

Assignment (19.3)

Name: B. Sarayu

Hall Ticket No: 2303A51842

Task 1 – Python to C++ Conversion

Prompt:

Convert a Python Student class into C++.

Python Code:

class Student:

```
    def __init__(self, name, marks):
```

```
        self.name = name
```

```
        self.marks = marks
```

```
    def display(self):
```

```
        print(self.name, self.marks)
```

C++ Code:

```
#include <iostream>
```

```
using namespace std;
```

```
class Student {
```

```
public:
```

```
    string name;
```

```
    int marks;
```

```
    Student(string n, int m){
```

```
        name = n; marks = m;
```

```
    }
```

```
    void display(){
```

```
        cout << name << " " << marks;
```

```
    }
```

```
};
```

Output:

Same output in both languages

The image shows a code editor with two tabs: `main.py` and `main.cpp`. The `main.py` tab is active, displaying a Python class `Student` with an `__init__` method and a `display` method. The `main.cpp` tab is also visible, showing the same class structure in C++ using `std::string` and `std::cout`. Both implementations create a `Student` object named `s1` with the name "Sarayu" and marks 95, and then call the `display` method. The output on the right shows "Name: Sarayu" and "Marks: 95".

```
main.py
1 class Student:
2     def __init__(self, name, marks):
3         self.name = name
4         self.marks = marks
5
6     def display(self):
7         print("Name:", self.name)
8         print("Marks:", self.marks)
9
10 # Test
11 s1 = Student("Sarayu", 95)
12 s1.display()
```

```
main.cpp
1 #include <iostream>
2 using namespace std;
3
4 class Student {
5 public:
6     string name;
7     int marks;
8
9     Student(string n, int m) {
10         name = n;
11         marks = m;
12     }
13
14     void display() {
15         cout << "Name: " << name << endl;
16         cout << "Marks: " << marks << endl;
17     }
18 };
19
20 int main() {
21     Student s1("Sarayu", 95);
22     s1.display();
23     return 0;
24 }
```

Output: Name: Sarayu
Marks: 95

Task 2 – Java to Python Conversion

Prompt:

Convert Java `isPrime()` method into Python.

Java Code:

```
boolean isPrime(int n){
    if(n<=1) return false;
```

```
for(int i=2;i<n;i++){  
    if(n%i==0) return false;  
}  
return true;  
}
```

Python Code:

```
def is_prime(n):  
    if n <= 1:  
        return False  
    for i in range(2, n):  
        if n % i == 0:  
            return False  
    return True
```

Output:

Input: 7 → True

Input: 4 → False

The image shows a code editor with two tabs: 'main.py' and 'Main.java'. The 'main.py' tab is active, displaying a Python function 'is_prime(n)' that checks if a number is prime. The function returns 'True' for prime numbers and 'False' for non-prime numbers. The code is as follows:

```
1- def is_prime(n):
2-     if n <= 1:
3-         return False
4-     for i in range(2, n):
5-         if n % i == 0:
6-             return False
7-     return True
8- print(is_prime(7))
9- print(is_prime(4))
```

The 'Main.java' tab is also visible, showing a Java implementation of the same logic. The code is as follows:

```
1- public class Main {
2-     public static boolean isPrime(int n) {
3-         if (n <= 1) return false;
4-
5-         for (int i = 2; i < n; i++) {
6-             if (n % i == 0)
7-                 return false;
8-         }
9-         return true;
10    }
11
12    public static void main(String[] args) {
13        System.out.println(isPrime(7));
14        System.out.println(isPrime(4));
15    }
16 }
```

The output of the Python code is shown on the right side of the editor, displaying 'True' and 'False' for the respective inputs. The output of the Java code is also shown, displaying 'true' and 'false' for the respective inputs.

Task 3 – Pseudocode to Python

Prompt:

Convert bubble sort pseudocode to Python.

Python Code:

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr

print(bubble_sort([5,2,9,1]))
```

Output:

[1, 2, 5, 9]



The screenshot shows a code editor with a dark theme. On the left is a sidebar with icons for Python, R, a database, a file explorer, a terminal, and a back arrow. The main editor area is titled 'main.py' and contains the following Python code:

```
1 def bubble_sort(arr):
2     n = len(arr)
3     for i in range(n):
4         for j in range(0, n-i-1):
5             if arr[j] > arr[j+1]:
6                 arr[j], arr[j+1] = arr[j+1], arr[j]
7     return arr
8
9 # Test
10 print(bubble_sort([5, 2, 9, 1]))
```

At the top right of the editor are icons for a full screen view, settings, a share button, and a blue 'Run' button. To the right of the code editor is an 'Output' panel. It displays the result of the code execution: '[1, 2, 5, 9]'. Below this, it says '=== Code Exec'.

Task 4 – SQL to Pandas

Prompt:

Convert SQL query to Pandas.

SQL:

SELECT name, salary FROM employees WHERE salary > 50000

Python Code:

```
import pandas as pd
data = {"name":["A","B","C"],"salary":[40000,60000,70000]}
df = pd.DataFrame(data)
result = df[df["salary"] > 50000][["name","salary"]]
print(result)
```

Output:

B 60000

C 70000



The screenshot shows a Jupyter Notebook interface with a dark theme. On the left is a sidebar with icons for file explorer, search, and other tools. The main area is divided into two panes. The left pane, titled 'main.py', contains the following Python code:

```
1 import pandas as pd
2
3 data = {
4     "name": ["A", "B", "C"],
5     "salary": [40000, 60000, 70000]
6 }
7
8 df = pd.DataFrame(data)
9
10 result = df[df["salary"] > 50000][["name", "salary"]]
11
12 print(result)
```

The right pane, titled 'Output', displays the result of the code execution as a table:

	name	salary
1	B	60000
2	C	70000

Below the table, the text '=== Code Execution Success' is visible.

Task 5 – Algorithm Translation Across Languages

Prompt:

Convert Python linear search to C.

Python Code:

```
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1
```

C Code:

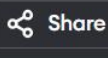
```
#include <stdio.h>
int linearSearch(int arr[], int n, int key){
    for(int i=0;i<n;i++){
        if(arr[i]==key) return i;
    }
    return -1;
}
```

Challenges:

- Syntax differences
- Memory management
- Library support



main.py



Run

```
1 def linear_search(arr, key):
2     for i in range(len(arr)):
3         if arr[i] == key:
4             return i
5     return -1
6
7 # Test
8 print(linear_search([10, 20, 30, 40], 30))
```

2

=== Co



main.c



Run

```
1 #include <stdio.h>
2
3 int linearSearch(int arr[], int n, int key) {
4     for(int i = 0; i < n; i++) {
5         if(arr[i] == key)
6             return i;
7     }
8     return -1;
9 }
10
11 int main() {
12     int arr[] = {10, 20, 30, 40};
13     int result = linearSearch(arr, 4, 30);
14
15     printf("Index: %d", result);
16     return 0;
17 }
```

Index: 2

=== Code Exe